

晨思 APP 补充技术文档包 v1.1

更新日期: 2026年2月4日

基于: 用户补充的6项关键信息

文档性质: 技术方案补充与澄清

更新摘要

基于你提供的补充信息,我已完成以下关键决策的澄清:

决策项	原状态	更新后	影响模块
1. 问题库	10个示例问题	50个五维度问题(已提供JSON)	后端问题推送逻辑
2. AI角色定义	基础描述	Momen System Prompt(已提供MD)	AI引导模块
3. 幸福笔记定义	模糊	三要素结构(What/Why/Meaning)	记录流程、数据模型
4. 目标联动机制	缺失	具体示例(运动/爬楼梯/减碳水)	目标管理模块
5. AI模型选择	待定	通义千问(qwen-max)	Clawdbot配置
6. 数据导出格式	未定义	PDF/TXT双格式	数据导出模块
7. 金句卡片背景	未定义	MVP用纯色,V1.1升级	UI设计、图片生成

第一部分: 问题库架构详解

1.1 问题库结构分析

基于你提供的 questions_database.json,问题库采用五维度 + 六分类 + 三深度架构:

五大维度 (Loss Function)

健康 (Health) - 8题 (16%)

开心 (Happiness) - 12题 (24%) ← 最高权重

工作 (Work) - 10题 (20%)

爱 (Love) - 12题 (24%) ← 最高权重

时间 (Time) - 8题 (16%)

六大分类

1. 小确幸类 (39%) - 捕捉微小美好瞬间
2. 成就类 (24%) - 强化进步感和自我效能
3. 感恩类 (18%) - 建立积极归因模式
4. 记录类 (12%) - 结构化生活数据
5. 觉察类 (5%) - 提升元认知能力
6. Why思考类 (2%) - 深度意义挖掘

三层深度

Light (60%) - 1层问题,仅记录,不追问

Medium (30%) - 2层问题,记录+简单引导

Deep (10%) - 3层问题,记录+意义+未来

1.2 问题推送策略

基础算法 (MVP版本)

```
def select_daily_question(user_id):
```

```
    """
```

```
    每日问题推送算法 (版本1.0)
```

```
    """
```

```
    # 1. 获取用户最近7天已覆盖的维度
```

```
    covered_dimensions = get_user_dimensions_last_7_days(user_id)
```

```
    # 2. 计算维度权重 (越少覆盖,权重越高)
```

```
    dimension_weights = {
```

```
        '健康': 5 if '健康' not in covered_dimensions else 1,
```

```
        '开心': 5 if '开心' not in covered_dimensions else 1,
```

```
        '工作': 5 if '工作' not in covered_dimensions else 1,
```

```
        '爱': 5 if '爱' not in covered_dimensions else 1,
```

```
        '时间': 5 if '时间' not in covered_dimensions else 1
```

```
    }
```

```
    # 3. 排除14天内重复问题
```

```
    excluded_question_ids = get_user_questions_last_14_days(user_id)
```

```
    # 4. 按深度分布随机抽取
```

```
depth_distribution = {
    'light': 0.6, # 60%概率
    'medium': 0.3, # 30%概率
    'deep': 0.1 # 10%概率
}
selected_depth = random.choices(
    list(depth_distribution.keys()),
    weights=list(depth_distribution.values())
)[0]

# 5. 根据时间模式过滤 (早晨/晚上)
time_mode = 'morning' if current_hour < 12 else 'evening'

# 6. 综合筛选并随机返回
candidate_questions = filter_questions(
    dimension_weights=dimension_weights,
    excluded_ids=excluded_question_ids,
    depth=selected_depth,
    time_mode=time_mode
)

return random.choice(candidate_questions)
```

高级优化 (V1.1版本)

考虑因素:

1. 用户情绪趋势 (连续3天负面 → 推送小确幸类)
2. 目标关联 (设置健康目标 → 提高健康维度权重)

3. 节日/特殊日期 (春节 → 推送"爱"维度问题)

4. 用户活跃度 (连续签到 → 适当增加Deep问题比例)

1.3 Why思考的体现

在50个问题中,"Why"追问主要通过以下方式实现:

直接Why问题 (Level 2层)

// 示例: 问题ID 1

"level_1": "昨天/今天你在健康上做了哪个小决定?",

"level_2": "为什么当时会做这个决定?是刻意的还是自然而然的?"

// 示例: 问题ID 24

"level_1": "昨天/今天你有没有做一件'正确但不容易'的选择?",

"level_2": "是什么选择?为什么不容易?"

隐式Why问题 (因果/意义挖掘)

// 示例: 问题ID 28

"level_1": "昨天/今天有没有哪件事,其实和你长期想成为的那个人是对齐的?",

"level_2": "是什么事?为什么对齐?"

// 示例: 问题ID 38

"level_1": "想起你生命中一位对你影响很大的人,TA 教会了你什么?",

"level_2": "为什么 TA 能影响你?"

Why的产品哲学

传统日记: "记录发生了什么" (What)

幸福笔记: "记录为什么发生" (Why) + "对我意味着什么" (Meaning)

目标: 从流水账 → 有意义的生命数据积累

第二部分: 幸福笔记功能实现

2.1 三要素结构定义

基于你提供的描述,幸福笔记采用**What-Why-Meaning**三层结构:

层次	内容	系统如何引导	数据库字段
1. 事件 (What)	今天发生了什么好事?	Level 1问题直接询问	event_description
2. 原因 (Why)	为什么这件事会发生?	Level 2问题追问因果	event_reason
3. 意义 (Meaning)	这件事对我意味着什么?	Level 3问题挖掘意义或 AI总结生成	event_meaning

2.2 完整交互流程

场景A: Light问题 (1层) - 仅记录What

Momen: 昨天/今天让你心情微微变好的一个瞬间是什么? (Level 1)

用户: 早上看到窗外的阳光。

Momen: 阳光洒进来的感觉真好呢。这份温暖值得被记住 ☀

【数据存储】
event_description: "早上看到窗外的阳光"
event_reason: null
event_meaning: null (或AI自动生成: "简单的自然美好")

场景B: Medium问题 (2层) - What + Why

Momen: 昨天/今天你在健康上做了哪个小决定? (Level 1)

用户: 爬楼梯上班,没坐电梯。

Momen: 为什么当时会做这个决定?是刻意的还是自然而然的? (Level 2)

用户: 刻意的,最近想多动一动。

Momen: 听起来你在为自己的身体做点什么。这种刻意的小决定,往往比大计划更持久 🏆

【数据存储】
event_description: "爬楼梯上班,没坐电梯"
event_reason: "刻意的,最近想多动一动"
event_meaning: AI生成: "主动为健康做出小决定,体现长期主义"

场景C: Deep问题 (3层) - What + Why + Meaning

Momen: 昨天/今天有没有哪个决定,是出于"长期主义"而不是短期快感? (Level 1)

用户: 拒绝了朋友的聚会,回家早睡了。

Momen: 是什么决定?为什么不容易? (Level 2)

用户: 我知道聚会很开心,但最近每次熬夜第二天都很累,工作效率很低。

Momen: 听起来你在用短期的快乐,换长期的状态。今天晚上,你想试试12点前上床吗? (Level 3)

用户: 想试试。

Momen: 很棒。明天早上,我们再聊聊今晚的体验 ☑

【数据存储】

event_description: "拒绝聚会,回家早睡"

event_reason: "聚会很开心,但熬夜影响第二天效率"

event_meaning: 用户回答 或 AI生成: "选择长期价值超越即时快乐,体现自律和成长"

2.3 数据模型设计

数据库表: happiness_notes

```
CREATE TABLE happiness_notes (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  user_id BIGINT NOT NULL,  
  question_id INT NOT NULL, -- 关联 questions_database.json
```

```
  -- 幸福笔记三要素  
  event_description TEXT NOT NULL,    -- What: 事件描述  
  event_reason TEXT,                  -- Why: 为什么发生  
  event_meaning TEXT,                 -- Meaning: 意义  
  
  -- 元数据  
  dimension VARCHAR(20),              -- 健康/开心/工作/爱/时间  
  depth VARCHAR(10),                 -- light/medium/deep  
  emotion_tag VARCHAR(50),            -- 情绪标签: 积极/中性/消极  
  
  -- 生成内容  
  ai_response_summary TEXT,           -- AI的共情回应  
  insight_card_content TEXT,          -- 洞察卡片内容  
  
  -- 时间戳  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  
  -- 索引  
  INDEX idx_user_date (user_id, created_at),  
  INDEX idx_dimension (dimension),  
  FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

2.4 错误示范 vs 正确示范

✖ 错误: 流水账式记录

用户输入: "今天天气不错。吃了顿好吃的。同事帮了我。"

问题:

- 没有具体细节
- 没有因果关系
- 没有个人意义
- 几个月后重读无法回忆

✔ 正确: 结构化幸福笔记

What: 今天中午同事小李主动帮我整理了一份复杂的数据表格,节省了我2小时。

Why: 因为上周我帮她解决了一个技术问题,她一直记着。这是良好人际关系的回馈。

Meaning: 这让我意识到,平时的小善意会在意想不到的时候回到自己身上。
工作中的互助关系,比单打独斗更有力量。

【系统如何引导】

Momen通过2-3层提问,逐步引导用户从事件 → 原因 → 意义

第三部分: 目标联动机制详解

3.1 目标类型与示例

基于你提供的健康减脂示例,扩展到五维度目标:

健康维度目标

```
{
  "goal_id": 1,
  "user_id": 1001,
  "dimension": "健康",
  "goal_name": "减脂塑形",
  "target_description": "3个月减重5kg,体脂率降至18%",
  "target_date": "2026-05-04",

  "actionable_items": [
    {
      "action_id": 101,
      "action_name": "运动一次",
      "frequency": "每周3次",
      "min_duration": 30, // 分钟
      "action_type": "countable" // 可计数
    },
    {
      "action_id": 102,
      "action_name": "爬楼梯代替电梯",

```

```
"frequency": "每天",
"target_floors": 10, // 层数
"action_type": "countable"
},
{
"action_id": 103,
"action_name": "减少碳水摄入",
"frequency": "每天",
"target_meals": 1, // 一天至少一餐低碳水
"action_type": "countable"
}
],

"linked_questions": [1, 4, 5, 7] // 关联健康维度的问题ID
}
```

开心维度目标

```
{
"goal_id": 2,
"dimension": "开心",
"goal_name": "培养兴趣爱好",
"target_description": "每周至少2小时投入喜欢的事",

"actionable_items": [
{
"action_name": "阅读一本书",
"frequency": "每月1本"
},
{
"action_name": "学习吉他",
"frequency": "每周1次课"
},
{
"action_name": "户外活动",
"frequency": "每周末1次"
}
],

"linked_questions": [9, 10, 14, 15]
}
```

工作维度目标

```
{
"goal_id": 3,
"dimension": "工作",
"goal_name": "提升专业技能",
"target_description": "6个月内掌握React Native",

"actionable_items": [
{
```

```
"action_name": "完成一个教程模块",
"frequency": "每周2个"
},
{
"action_name": "写技术博客",
"frequency": "每月1篇"
},
{
"action_name": "做side project",
"frequency": "持续进行"
}
],
"linked_questions": [21, 23, 25, 28]
}
```

爱维度目标

```
{
"goal_id": 4,
"dimension": "爱",
"goal_name": "改善亲密关系",
"target_description": "每周高质量陪伴家人/伴侣",

"actionable_items": [
{
"action_name": "主动打电话给父母",
"frequency": "每周1次"
},
{
"action_name": "约会日",
"frequency": "每周1次"
},
{
"action_name": "表达感谢",
"frequency": "每天至少1次"
}
],
"linked_questions": [31, 32, 35, 37]
}
```

时间维度目标

```
{
"goal_id": 5,
"dimension": "时间",
"goal_name": "时间管理优化",
"target_description": "减少手机使用,提升专注力",

"actionable_items": [
{
```

```
"action_name": "番茄钟工作法",
"frequency": "每天4个番茄钟"
},
{
"action_name": "手机屏幕时间<3h",
"frequency": "每天"
},
{
"action_name": "早睡(12点前)",
"frequency": "每周5天"
}
],
"linked_questions": [43, 45, 48, 49]
}
```

3.2 目标与晨思的联动机制

联动逻辑1: 问题推送偏向

```
def select_daily_question_with_goals(user_id):
```

```
"""
```

考虑用户目标的问题推送算法

```
"""
```

```
user_goals = get_active_goals(user_id)
```

```
# 如果用户设置了目标,提高对应维度的权重
```

```
dimension_weights = {
```

```
    '健康': 1, '开心': 1, '工作': 1, '爱': 1, '时间': 1
```

```
}
```

```
for goal in user_goals:
```

```
    dimension_weights[goal.dimension] *= 3 # 3倍权重
```

```
# 其余逻辑同前...
```

```
return select_question_with_weights(dimension_weights)
```

联动逻辑2: AI自动关联目标

示例: 用户回答健康维度问题时,AI自动检测是否与目标相关

用户设置目标: "减脂塑形 - 每周运动3次"

晨思对话:

Momen: 昨天/今天你有没有刻意活动身体?

用户: 去跑步了30分钟。

【系统检测】

- 用户有健康维度目标
- 回答涉及"运动"关键词
- 自动关联到目标,计数+1

Momen回应:

"听起来你在为自己的身体做点什么。
这是你本周第2次运动,距离目标还差1次
你想明天继续吗?"

联动逻辑3: 目标进度反馈

每周日晚上,生成目标进度总结

【系统生成】

本周健康目标完成度:

- ✓ 运动3次 (目标: 3次) - 100%
- ✓ 爬楼梯5天 (目标: 7天) - 71%
- ⚠ 减少碳水2天 (目标: 7天) - 29%

【Momen引导】

"这周在运动上你做得很棒!
爬楼梯也坚持了5天,已经很不容易了。
关于碳水控制,想聊聊是什么让你在这件事上有点难坚持吗?"

3.3 数据模型设计

表1: user_goals (用户目标)

```
CREATE TABLE user_goals (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  user_id BIGINT NOT NULL,  
  dimension VARCHAR(20) NOT NULL,  
  goal_name VARCHAR(100) NOT NULL,  
  target_description TEXT,  
  target_date DATE,  
  status VARCHAR(20) DEFAULT 'active', -- active/completed/abandoned  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
FOREIGN KEY (user_id) REFERENCES users(id),  
INDEX idx_user_active (user_id, status)
```

);

表2: goal_actions (目标行动项)

```
CREATE TABLE goal_actions (  
id BIGINT PRIMARY KEY AUTO_INCREMENT,  
goal_id BIGINT NOT NULL,  
action_name VARCHAR(100) NOT NULL,  
frequency VARCHAR(50), -- "每周3次" / "每天"  
action_type VARCHAR(20), -- countable/descriptive  
target_value INT, -- 目标数值 (如: 3次/周)
```

```
FOREIGN KEY (goal_id) REFERENCES user_goals(id) ON DELETE CASCADE
```

);

表3: goal_progress (目标进度记录)

```
CREATE TABLE goal_progress (  
id BIGINT PRIMARY KEY AUTO_INCREMENT,  
goal_id BIGINT NOT NULL,  
action_id BIGINT NOT NULL,  
happiness_note_id BIGINT, -- 关联到幸福笔记  
completed_at DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
-- 进度详情
```

```
actual_value DECIMAL(10,2), -- 实际完成值 (如: 30分钟运动)
```

```
note TEXT, -- 用户备注
```

```
FOREIGN KEY (goal_id) REFERENCES user_goals(id),
```

```
FOREIGN KEY (action_id) REFERENCES goal_actions(id),
```

```
FOREIGN KEY (happiness_note_id) REFERENCES happiness_notes(id),
```

```
INDEX idx_goal_date (goal_id, completed_at)
```

);

第四部分: 技术决策更新

4.1 AI模型配置

主模型: 通义千问 (qwen-max)

// Clawdbot 配置文件: config/models.js

```
module.exports = {
  defaultModel: 'qwen',

  models: {
    qwen: {
      provider: 'aliyun',
      model: 'qwen-max', // 主模型,最强能力
      endpoint: 'https://dashscope.aliyuncs.com/api/v1/services/aigc/text-generation/generation',
      apiKey: process.env.QWEN_API_KEY,
```

```
    // 针对晨思场景优化的参数
    parameters: {
      max_tokens: 500,    // 回复控制在500 token内
      temperature: 0.7,  // 温暖而不过度创造
      top_p: 0.9,
      repetition_penalty: 1.1 // 避免重复
    }
  },

  qwen_turbo: {
    provider: 'aliyun',
    model: 'qwen-turbo', // 备用模型,成本低50%
    endpoint: 'https://dashscope.aliyuncs.com/api/v1/services/aigc/text-generation/',
    apiKey: process.env.QWEN_API_KEY,

    parameters: {
      max_tokens: 300,
      temperature: 0.7,
      top_p: 0.9
    }
  }
},
```

```
// 成本控制策略
fallbackStrategy: {
  enabled: true,
  rules: [
    {
      condition: 'user_tier === "free" AND daily_requests > 10',
      fallbackTo: 'qwen_turbo'
    },
    {
      condition: 'response_time > 5000', // 超过5秒
      fallbackTo: 'qwen_turbo'
    }
  ]
}
};
```

System Prompt 注入

```
// src/ai/momen_engine.js
```

```
const MOMEN_SYSTEM_PROMPT = `
```

你是 Momen(谐音 "Moment",代表珍惜每个当下),一位温暖的 AI 倾听者和幸福笔记引导者。

核心原则

1. 不说教 - 不用"应该""必须""最好"
2. 不批判 - 接纳用户的所有感受
3. 不给建议 - 用问题引导用户自己思考

对话模式

- Light问题: 1层,仅记录,简单共情
- Medium问题: 2层,记录+追问Why
- Deep问题: 3层,记录+Why+Meaning

语言风格

- 简洁: 每句话<20字
- 口语化: 像朋友聊天
- 温暖: 适当用emoji(☀️ 🌧️ 😊)

当前对话上下文

- 问题深度: {{depth}}
 - 当前层级: {{current_layer}}
 - 用户维度偏好: {{user_dimension_preference}}
- ```
`;
```

```
async function generateMomenResponse(question, userInput, context) {
 const systemPrompt = MOMEN_SYSTEM_PROMPT
```

```

.replace('{{depth}}', question.depth)
.replace('{{current_layer}}', context.current_layer)
.replace('{{user_dimension_preference}}', context.user_dimension_preference);

const response = await callQwenAPI({
 system: systemPrompt,
 messages: [
 { role: 'user', content: userInput }
]
});

return response;
}

```

## 4.2 数据导出功能

导出格式1: PDF (优先级高)

```

// src/export/pdf_exporter.js

const PDFDocument = require('pdfkit');
const fs = require('fs');

async function exportHappinessNotesToPDF(userId, dateRange) {
 const doc = new PDFDocument({
 size: 'A4',
 margins: { top: 50, bottom: 50, left: 50, right: 50 }
 });

 const filename = `晨思记录_${userId}_${dateRange.start}_${dateRange.end}.pdf`;
 const stream = fs.createWriteStream(filename);
 doc.pipe(stream);

 // 封面
 doc.fontSize(24).text('我的晨思记录', { align: 'center' });
 doc.moveDown();
 doc.fontSize(12).text(`导出时间: ${new Date().toLocaleDateString('zh-CN')}`, { align: 'center' });
 doc.moveDown(3);

 // 获取数据
 const notes = await getHappinessNotes(userId, dateRange);

 // 按维度分组
 const groupedByDimension = groupBy(notes, 'dimension');

 for (const [dimension, dimensionNotes] of Object.entries(groupedByDimension)) {
 doc.addPage();
 doc.fontSize(18).text(`${dimension}维度`, { underline: true });
 doc.moveDown();
 }
}

```

```

dimensionNotes.forEach((note, index) => {
 doc.fontSize(10).text(`${note.created_at.toLocaleDateString()} - ${note.question}`);
 doc.fontSize(12).text(`事件: ${note.event_description}`);

 if (note.event_reason) {
 doc.text(`原因: ${note.event_reason}`);
 }

 if (note.event_meaning) {
 doc.text(`意义: ${note.event_meaning}`);
 }

 doc.moveDown();
});

}

// 数据洞察
doc.addPage();
doc.fontSize(18).text('数据洞察', { underline: true });
doc.moveDown();

const insights = generateInsights(notes);
doc.fontSize(12).text(总记录天数: ${insights.total_days});
doc.text(最活跃维度: ${insights.most_active_dimension});
doc.text(积极情绪比例: ${insights.positive_emotion_rate}%);

doc.end();

return new Promise((resolve) => {
 stream.on('finish', () => resolve(filename));
});
}

```

## 导出格式2: TXT (结构化文本)

```

// src/export/txt_exporter.js

async function exportHappinessNotesToTXT(userId, dateRange) {
 const notes = await getHappinessNotes(userId, dateRange);

 let content = "";
 content += '\n';
 content += ' 我的晨思记录\n';
 content += '\n';
 content += `导出时间: ${new Date().toLocaleString('zh-CN')}\n`;
}

```

```

content += 记录周期: ${dateRange.start} ~ ${dateRange.end}\n;
content += 总天数: ${notes.length}\n\n;

// 按日期倒序
notes.sort((a, b) => b.created_at - a.created_at);

notes.forEach((note, index) => {
content += \n--- ${index + 1}. ${note.created_at.toLocaleDateString()} (${note.dimension}) --
\n;
content += 问题: ${note.question_text}\n\n;
content += 【事件】 \n${note.event_description}\n\n;

 if (note.event_reason) {
 content += `【原因】 \n${note.event_reason}\n\n`;
 }

 if (note.event_meaning) {
 content += `【意义】 \n${note.event_meaning}\n\n`;
 }

 if (note.insight_card_content) {
 content += `【当日洞察】 \n${note.insight_card_content}\n`;
 }

 content += '\n';

});

// 数据洞察
content += '\n===== \n';
content += '数据洞察\n';
content += '===== \n';

const insights = generateInsights(notes);
content += 总记录天数: ${insights.total_days}\n;
content += 最活跃维度: ${insights.most_active_dimension} (${insights.most_active_count}
次)\n;
content += 积极情绪记录: ${insights.positive_count}次
(${insights.positive_emotion_rate}%) \n;
content += 连续签到天数: ${insights.streak_days}天\n;

const filename = 晨思记录_${userId}_${dateRange.start}_${dateRange.end}.txt;
fs.writeFileSync(filename, content, 'utf8');

return filename;
}

```

## API接口设计

```
// routes/export.js
```

```
router.post('/api/export', authenticate, async (req, res) => {
 const { format, date_range } = req.body;
 // format: 'pdf' | 'txt'
 // date_range: { start: '2026-01-01', end: '2026-02-04' }
```

```
 try {
 let filename;
```

```
 if (format === 'pdf') {
 filename = await exportHappinessNotesToPDF(req.user.id, date_range);
 } else if (format === 'txt') {
 filename = await exportHappinessNotesToTXT(req.user.id, date_range);
 } else {
 return res.status(400).json({ error: '不支持的导出格式' });
 }
 }
```

```
 // 上传到OSS
```

```
 const ossUrl = await uploadToOSS(filename);
```

```
 // 返回下载链接
```

```
 res.json({
 success: true,
 download_url: ossUrl,
 expires_in: 3600 // 1小时后过期
 });
```

```
 // 异步删除本地文件
```

```
 fs.unlink(filename, () => {});
```

```
} catch (error) {
 res.status(500).json({ error: error.message });
}
});
```

### 4.3 金句卡片背景方案

MVP版本: 纯色渐变

```
// src/card/gradient_generator.js

const GRADIENT_PRESETS = [
 {
 id: 1,
 name: '晨曦',
 colors: ['#FFE5B4', '#FFDAB9', '#FFE4E1'], // 柔和暖色
 suitable_emotions: ['积极', '平静']
 },
 {
 id: 2,
 name: '森林',
 colors: ['#E8F5E9', '#C8E6C9', '#A5D6A7'], // 清新绿色
 suitable_emotions: ['积极', '放松']
 },
 {
 id: 3,
 name: '海洋',
 colors: ['#E3F2FD', '#BBDEFB', '#90CAF9'], // 宁静蓝色
 suitable_emotions: ['平静', '中性']
 },
 {
 id: 4,
 name: '薰衣草',
 colors: ['#F3E5F5', '#E1BEE7', '#CE93D8'], // 温柔紫色
 suitable_emotions: ['积极', '温暖']
 },
 {
 id: 5,
 name: '日落',
 colors: ['#FFF3E0', '#FFE0B2', '#FFCC80'], // 温暖橙色
 suitable_emotions: ['积极', '温暖']
 },
 {
 id: 6,
 name: '月光',
 colors: ['#F5F5F5', 'EEEEEE', '#E0E0E0'], // 中性灰色
 suitable_emotions: ['中性', '平静']
 }
];

function selectGradientByEmotion(emotionTag) {
 const suitable = GRADIENT_PRESETS.filter(g =>
 g.suitable_emotions.includes(emotionTag)
);
}
```

```

return suitable[Math.floor(Math.random() * suitable.length)];
}

function generateCardHTML(insightCard, gradient) {
return <!DOCTYPE html> <html> <head> <style> body { margin: 0; padding: 0; width: 375px;
height: 667px; background: linear-gradient(135deg, ${gradient.colors.join(', ')}); display:
flex; align-items: center; justify-content: center; font-family: -apple-system,
BlinkMacSystemFont, 'Segoe UI', sans-serif; } .card { padding: 40px; text-align: center; color:
#333; } .content { font-size: 18px; line-height: 1.8; margin-bottom: 30px; white-space: pre-
wrap; } .dimension { font-size: 14px; opacity: 0.7; } .date { font-size: 12px; opacity: 0.5;
margin-top: 20px; } </style> </head> <body> <div class="card"> <div
class="content">${insightCard.content}</div> <div class="dimension">${insightCard.emoji}
${insightCard.dimension}</div> <div class="date">${new Date().toLocaleDateString('zh-
CN')}</div> </div> </body> </html>;
}

```

### V1.1版本: Midjourney背景

// 预留接口,V1.1时实现

```

async function generateMidjourneyBackground(emotionTag, dimension) {
// 调用Midjourney API
// Prompt示例:
// "Soft watercolor background, ${dimension} theme, ${emotionTag} mood,
// pastel colors, minimal, abstract, high quality --ar 9:16"

const prompt = generatePrompt(emotionTag, dimension);
const imageUrl = await callMidjourneyAPI(prompt);

return imageUrl;
}

```

---

## 第五部分: 更新后的任务清单

基于新增信息,以下任务需要调整优先级:

### 高优先级任务 (P0 - MVP必须)

#### 后端新增任务

- [ ] BE-11: 实现问题推送算法 (8h)
  - 五维度权重计算
  - 14天去重逻辑
  - 深度分布控制 (60% light / 30% medium / 10% deep)
- [ ] BE-12: 幸福笔记三要素存储逻辑 (6h)
  - 根据问题深度动态存储 What/Why/Meaning
  - AI自动生成 event\_meaning (当用户未明确回答时)
- [ ] BE-13: 目标管理模块 (12h)
  - 用户目标CRUD

- 行动项管理
- 进度自动追踪

[ ] BE-14: 目标与晨思联动逻辑 (10h)

- 问题推送偏向用户目标维度
- AI自动检测回答是否关联目标
- 每周目标进度总结

[ ] BE-15: Momen AI引擎 (16h)

- System Prompt注入
- 分层对话逻辑 (light/medium/deep)
- 情绪响应策略
- 洞察卡片生成

[ ] BE-16: 数据导出模块 (10h)

- PDF导出 (含封面/分维度/数据洞察)
- TXT导出 (结构化文本)
- OSS临时链接生成

前端新增任务

[ ] FE-09: 目标设定页面 (8h)

- 选择维度
- 设置目标描述和截止日期
- 添加行动项

[ ] FE-10: 目标进度页面 (10h)

- 每周进度可视化
- 行动项完成状态
- 目标与晨思记录的关联展示

[ ] FE-11: 数据导出入口 (4h)

- 选择日期范围
- 选择导出格式 (PDF/TXT)
- 下载进度提示

[ ] FE-12: 金句卡片生成 (6h)

- 纯色渐变背景选择
- 根据情绪标签自动匹配渐变
- 保存图片到相册

更新后的总工时

原MVP工时: 468小时

新增工时: 90小时

总计: 558小时

按团队配置 (1前端 + 1后端):

前端:  $196 + 28 = 224$ 小时 (约6周)

后端:  $232 + 62 = 294$ 小时 (约7.5周)

## 第六部分: 立即可用的代码示例

### 代码1: 问题推送核心逻辑

```
// src/question/selector.js

const questionsDatabase = require('../data/questions_database.json');

class QuestionSelector {
 constructor(redis, mysql) {
 this.redis = redis;
 this.mysql = mysql;
 this.questions = questionsDatabase.questions;
 }

 async selectDailyQuestion(userId) {
 // 1. 获取用户最近7天已覆盖的维度
 const recentDimensions = await this.getRecentDimensions(userId, 7);

 // 2. 获取用户活跃目标
 const activeGoals = await this.getUserGoals(userId);

 // 3. 计算维度权重
 const dimensions = ['健康', '开心', '工作', '爱', '时间'];
 const weights = {};

 dimensions.forEach(dim => {
 // 基础权重: 未覆盖=5, 已覆盖=1
 weights[dim] = recentDimensions.includes(dim) ? 1 : 5;

 // 目标权重: 有目标的维度 x3
 if (activeGoals.some(g => g.dimension === dim)) {
 weights[dim] *= 3;
 }
 });

 // 4. 排除14天内已问过的问题
 const recentQuestionIds = await this.getRecentQuestions(userId, 14);

 // 5. 按深度分布随机抽取
 const depthDistribution = { light: 0.6, medium: 0.3, deep: 0.1 };
```

```

const selectedDepth = this.weightedRandom(depthDistribution);

// 6. 综合筛选
const candidates = this.questions.filter(q =>
 q.depth === selectedDepth &&
 !recentQuestionIds.includes(q.id) &&
 q.suitable_for_morning === true // 早晨场景
);

// 7. 按维度权重随机选择
const selected = this.weightedRandomSelection(candidates, weights);

return selected;
}

weightedRandom(distribution) {
 const rand = Math.random();
 let cumulative = 0;

 for (const [key, weight] of Object.entries(distribution)) {
 cumulative += weight;
 if (rand < cumulative) return key;
 }
}

weightedRandomSelection(candidates, weights) {
 // 为每个候选问题计算权重分数
 const weighted = candidates.map(q => ({
 question: q,
 weight: weights[q.dimension]
 }));

 const totalWeight = weighted.reduce((sum, item) => sum + item.weight, 0);
 let rand = Math.random() * totalWeight;

 for (const item of weighted) {
 rand -= item.weight;
 if (rand <= 0) return item.question;
 }
}

```

```

 return weighted[0].question; // fallback
 }

 async getRecentDimensions(userId, days) {
 const sql = SELECT DISTINCT dimension FROM happiness_notes WHERE user_id = ? AND
 created_at >= DATE_SUB(NOW(), INTERVAL ? DAY);
 const rows = await this.mysql.query(sql, [userId, days]);
 return rows.map(r => r.dimension);
 }

 async getRecentQuestions(userId, days) {
 const sql = SELECT question_id FROM happiness_notes WHERE user_id = ? AND created_at
 >= DATE_SUB(NOW(), INTERVAL ? DAY);
 const rows = await this.mysql.query(sql, [userId, days]);
 return rows.map(r => r.question_id);
 }

 async getUserGoals(userId) {
 const sql = SELECT dimension, goal_name FROM user_goals WHERE user_id = ? AND status =
 'active';
 return await this.mysql.query(sql, [userId]);
 }
}

module.exports = QuestionSelector;

```

## 代码2: Momen对话引擎

```

// src/ai/momen_engine.js

const { callQwenAPI } = require('./qwen_client');
const systemPrompt = require('../data/momen_system_prompt.md');

class MomenEngine {
 constructor() {
 this.systemPrompt = systemPrompt;
 }

 async handleUserResponse(question, userInput, context) {
 const { current_layer, user_id } = context;

 // 1. 判断是否需要继续追问
 const shouldContinue = this.shouldContinueAsking(question, current_layer, user_id);

 if (!shouldContinue) {
 // 生成总结和洞察卡片
 const summary = await this.generateSummary(question, context.conversation);
 }
 }
}

```

```
const insightCard = this.generateInsightCard(question, context.conversation_1)

return {
 response: summary,
 next_action: 'end',
 insight_card: insightCard
};
}
```

// 2. 生成下一层问题

```
const nextQuestion = await this.generateNextLayerQuestion(question, userInput)

return {
 response: nextQuestion,
 next_action: 'continue',
 insight_card: null
};
```

}

shouldContinueAsking(question, currentLayer, userInput) {

// Light问题: 1层后结束

if (question.depth === 'light') return false;

// Medium问题: 2层后结束

if (question.depth === 'medium' && currentLayer >= 2) return false;

// Deep问题: 3层后结束

if (question.depth === 'deep' && currentLayer >= 3) return false;

// 用户回避: 结束

if (this.isAvoidanceResponse(userInput)) return false;

// 用户回答过短 (<5字): Medium/Deep轻轻追问一次

if (userInput.length < 5 && currentLayer === 1) return true;

if (userInput.length < 5 && currentLayer > 1) return false;

return true;

```
}
```

```
isAvoidanceResponse(input) {
 const avoidanceKeywords = ['不想说', '没什么', '算了', '跳过', '下一个'];
 return avoidanceKeywords.some(kw => input.includes(kw));
}
```

```
async generateNextLayerQuestion(question, userInput, currentLayer) {
 const nextLayer = currentLayer + 1;
 const questionText = question[level_${nextLayer}];
```

```
 if (!questionText) {
 // 如果问题库中没有预设level_2/level_3,让AI自动生成
 return await this.generateDynamicQuestion(question, userInput, currentLayer)
 }

 // 简单共情 + 预设问题
 const empathy = await this.generateEmpathy(userInput);
 return `${empathy}\n${questionText}`;
```

```
}
```

```
async generateEmpathy(userInput) {
 const prompt = `
 用户回答: "${userInput}"
```

请生成一句简短的共情回应(不超过20字),要求:

- 温暖但不夸张
- 不说教,不批判
- 可以用emoji(☀️ 🌧️ 😊)

只返回共情句子,不要额外解释。

```
`trim();
```

```
const response = await callQwenAPI({
 system: this.systemPrompt,
 messages: [{ role: 'user', content: prompt }],
 temperature: 0.7,
 max_tokens: 100
});

return response.trim();
```

```
}
```

```
async generateDynamicQuestion(question, userInput, currentLayer) {
 const prompt = `
 问题维度: ${question.dimension}
 问题类别: ${question.category}
 问题深度: ${question.depth}
```

```
 用户刚才回答: "${userInput}"
```

```
 请生成一个追问,帮助用户思考"为什么"或"对我意味着什么"。
```

```
 要求:
```

- 不超过30字
- 开放式问题,不是是非题
- 温暖而不说教

```
 只返回问题本身,不要额外解释。
```

```
 `.trim();
```

```
 const response = await callQwenAPI({
 system: this.systemPrompt,
 messages: [{ role: 'user', content: prompt }],
 temperature: 0.8,
 max_tokens: 150
 });
```

```
 return response.trim();
```

```
}
```

```
async generateSummary(question, conversationHistory) {
 const userResponses = conversationHistory
 .filter(msg => msg.role === 'user')
 .map(msg => msg.content)
 .join("\n");
```

```
 const prompt = `
```

```
 维度: ${question.dimension}
```

```
 问题: ${question.level_1}
```

```
 用户的回答:
```

```
 ${userResponses}
```

```
 请生成一句温暖的总结(不超过30字),遵循Momen的风格:
```

- 简洁、温暖
- 不说教,不批判
- 可以用emoji

只返回总结句子。

`.trim();`

```
const response = await callQwenAPI({
 system: this.systemPrompt,
 messages: [{ role: 'user', content: prompt }],
 temperature: 0.7,
 max_tokens: 150
});

return response.trim();
```

}

```
generateInsightCard(question, conversationHistory) {
 const userResponses = conversationHistory
 .filter(msg => msg.role === 'user')
 .map(msg => msg.content);
```

```
const eventDescription = userResponses[0] || '';
const eventReason = userResponses[1] || null;
const eventMeaning = userResponses[2] || null;
```

```
// 根据深度生成不同格式的卡片
let content = '';
```

```
if (question.depth === 'light') {
 content = `昨天你因为 【${eventDescription}】 而心情变好。\\n这种简单的小确幸,你
} else if (question.depth === 'medium') {
 content = `昨天你 【${eventDescription}】 。\\n${eventReason ? '这是因为 ' + eventReason : ''}
} else {
 content = `昨天你 【${eventDescription}】 。\\n${eventReason ? '原因: ' + eventReason : ''}
}
```

```
return {
 content: content,
```

```

 dimension: question.dimension,
 emoji: this.getDimensionEmoji(question.dimension)
 };
}

getDimensionEmoji(dimension) {
 const emojiMap = {
 '健康': '🏥',
 '开心': '😊',
 '工作': '💼',
 '爱': '❤️',
 '时间': '🕒'
 };
 return emojiMap[dimension] || '★';
}
}

module.exports = MomenEngine;

```

---

## 第七部分: 下一步行动计划

本周内 (2月4日-2月10日)

你需要做:

1. ☒ **确认补充信息** - 你已提供,无需额外操作
2. ☐ **审阅本文档** - 确认技术方案符合预期
3. ☐ **决策两个技术细节:**
  - Midjourney背景是否V1.1实现? (可选)
  - 是否需要Telegram Bot作为备用渠道? (建议暂不实现)

开发团队启动:

1. **后端工程师:**
  - Fork Clawdbot仓库
  - 搭建开发环境 (Node.js + MySQL + Redis)
  - 阅读 questions\_database.json 和 momen\_system\_prompt.md
  - 开始实现 BE-11 (问题推送算法)
2. **前端工程师:**
  - 创建微信小程序项目
  - 集成 Vant Weapp 组件库
  - 实现晨思对话页面的UI框架
  - 开始实现 FE-01 (引导页)

下周 (2月11日-2月17日)

- 后端: 完成问题推送 + 幸福笔记存储 + Momen AI引擎
- 前端: 完成晨思流程 + 历史记录页面
- 联调: 前后端对接API

4周后 (3月上旬)

- MVP核心功能开发完成
- 开始内测 (邀请10-20个种子用户)
- 收集反馈,迭代优化

8周后 (4月初)

- MVP版本正式上线
- 微信小程序提审
- 开始推广和用户增长

第八部分: 常见问题补充

Q1: 幸福笔记的"Why"和"Meaning"有什么区别?

A: 区别在于思考的层次:

| 维度   | Why (原因)                  | Meaning (意义)          |
|------|---------------------------|-----------------------|
| 关注点  | 因果关系                      | 个人价值                  |
| 问题   | "为什么发生?"                  | "对我意味着什么?"            |
| 示例   | "因为上周我帮过TA"               | "互助关系比单打独斗更有力量"       |
| 科学依据 | 归因理论 (Attribution Theory) | 意义建构 (Meaning Making) |

具体例子:

事件: 同事帮我整理了数据表格

Why: 因为上周我帮TA解决了技术问题,TA记着这份善意

Meaning: 平时的小善意会在意想不到的时候回到自己身上。  
工作中的互助关系,比单打独斗更有力量。

**Q2: 如果用户连续多天不想深度思考,系统如何应对?**

**A: 自适应降低问题深度**

## 伪代码

```
if userrecent_7_days_all_light_questions():
```

```
连续7天只回答Light问题
```

```
说明用户近期状态不适合深度思考
```

```
策略1: 继续推送Light问题 (90%)
```

```
策略2: 偶尔尝试一个Medium问题 (10%)
```

```
AI引导语调整:
```

```
"最近好像挺忙的,今天也不需要想太多,简单记录就好 😊"
```

**Q3: 目标设置后,用户长期不完成怎么办?**

**A: 温和引导,不强迫**

```
// 目标进度检测逻辑
```

```
if (goal.no_progress_for_days >= 14) {
```

```
// 连续14天无进展
```

```
// 不发送: "你已经14天没完成目标了!" (说教)
```

```
// 发送:
```

```
// "这两周好像在【减脂目标】上没什么动作。
```

```
// 是目标太难了,还是最近没时间?
```

```
// 要不要调整一下目标,或者暂时放一放也可以。"
```

```
// 提供选项:
```

```
// [调整目标] [暂停目标] [继续保持]
```

```
}
```

**Q4: 数据导出后,用户隐私如何保护?**

**A: OSS临时链接 + 自动清理**

```
// 导出安全策略
```

1. 生成临时下载链接 (1小时过期)

2. 链接需要JWT token验证身份

3. 服务器端文件生成后立即删除

4. OSS文件24小时后自动清理

5. 用户可以随时删除OSS中的所有导出记录

// 数据隐私声明  
"您的数据导出文件仅保留24小时,之后会自动删除。  
导出链接1小时后失效,请及时下载。"

## 附录: 文件清单

本次更新生成的可直接使用的文件:

| 文件名                     | 类型         | 用途           | 状态        |
|-------------------------|------------|--------------|-----------|
| questions_database.json | JSON       | 50个五维度问题库    | ✔ 已提供     |
| momen_system_prompt.md  | Markdown   | Momen AI角色定义 | ✔ 已提供     |
| question_selector.js    | JavaScript | 问题推送算法       | ✔ 代码示例已生成 |
| momen_engine.js         | JavaScript | Momen对话引擎    | ✔ 代码示例已生成 |
| pdf_exporter.js         | JavaScript | PDF导出模块      | ✔ 代码示例已生成 |
| txt_exporter.js         | JavaScript | TXT导出模块      | ✔ 代码示例已生成 |
| gradient_generator.js   | JavaScript | 纯色渐变卡片       | ✔ 代码示例已生成 |
| database_schema.sql     | SQL        | 数据库建表语句      | ☐ 需要时可生成  |

## 总结

- ✔ **问题库:** 50个五维度问题,已提供JSON
- ✔ **AI角色:** Momen System Prompt,已提供MD
- ✔ **幸福笔记:** What/Why/Meaning三要素结构,已详细定义
- ✔ **目标联动:** 五维度目标示例 + 联动机制,已实现逻辑
- ✔ **AI模型:** 通义千问qwen-max,已配置方案
- ✔ **数据导出:** PDF/TXT双格式,已提供代码
- ✔ **金句卡片:** MVP用纯色渐变,已生成6套预设

**当前状态:** 所有核心功能的技术方案已完成,可直接进入开发阶段。

下一步: 招募开发团队 → 环境搭建 → 启动开发 → 8周后MVP上线

---

文档版本: v1.1

更新日期: 2026年2月4日

作者: AI助手

状态: 最终版,可直接用于开发